Chapter 8: Queues

Fundamentals of Python: Data Structures, Second Edition

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Learning Objectives

- Describe the features of a queue and the operations on it
- Choose a queue implementation based on its performance characteristics
- Recognize applications where it is appropriate to use a queue
- Explain the difference between a queue and a priority queue
- Recognize applications where it is appropriate to use a priority queue

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

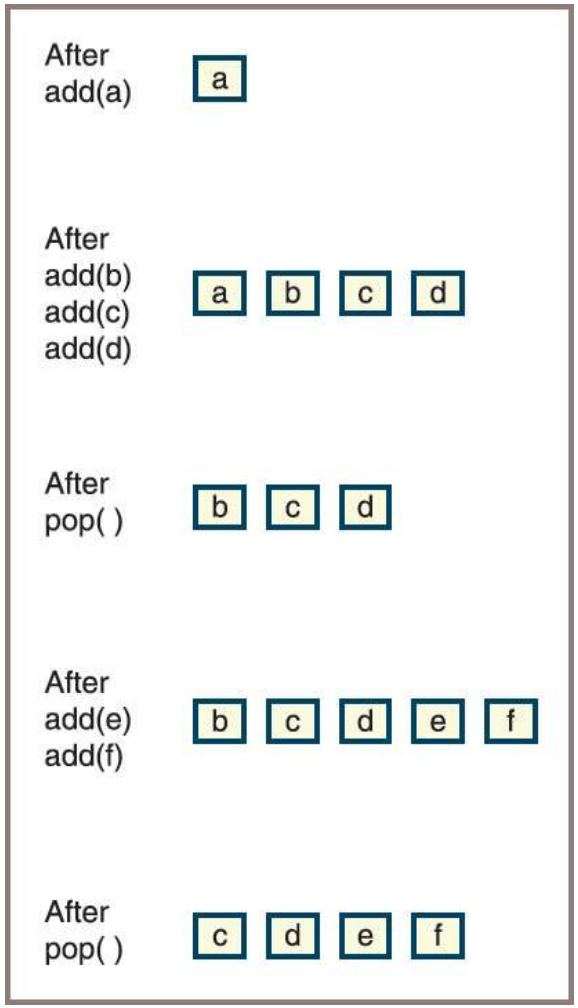
Overview of Queues (1 of 3)

- Queues are linear collections:
  - Insertions are restricted to one end, called the rear
  - Removals are restricted to the other end, called the front
- A queue supports a first-in-first-out (FIFO) protocol
- Queues have two fundamental operations:
  - **`add`** – adds an item to the rear of a queue
  - **`pop`** – removes an item from the front

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Overview of Queues (2 of 3)

Figure 8-1: The states in the lifetime of a queue



Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Overview of Queues (3 of 3)

- Initially, the queue is empty:
  - Then an item called **a** is added
  - Next, three more items called **b**, **c**, and **d** are added, after which an item is popped, and so forth
- Priority queue
  - Higher-priority items are popped before those of lower priority
  - Items of equal priority are popped in FIFO order
- Queue examples in computer science
  - **CPU access**—Processes are queued for access to a shared CPU
  - **Disk access**—Processes are queued for access to a shared secondary storage device
  - **Printer access**—Print jobs are queued for access to a shared laser printer

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

The Queue Interface and Its Use (1 of 5)

- You can use a Python list to emulate a queue:
  - Simplest strategy is to use the **`list`** method **`append`** to add an item to the rear of a queue
  - Use the **`list`** method **`pop(0)`** to remove and return the item at the front of its queue
- Drawback to this method
  - All the other list operations can manipulate the queue as well
- Aside from the **`add`** and **`pop`** operations,
  - It will be useful to have a **`peek`** operation, which returns the item at the front of the queue

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

The Queue Interface and Its Use (2 of 5)

- Table 8.1 The Methods in the Queue Interface

Queue Method	What It Does
`q.isEmpty()`	Returns `True` if `q` is empty or `False` otherwise.
`__len__(q)`	Same as `len(q)`. Returns the number of items in `q`.
`__str__(q)`	Same as `str(q)`. Returns the string representation of `q`.
`q.__iter__()`	Same as `iter(q)`, or `for item in q`. Visits each item in `q`, from front to rear.
`q.__contains__(item)`	Same as `item in q`. Returns `True` if item is in `q` or `False` otherwise.
`q1__add__(q2)`	Same as `q1 + q2`. Returns a new queue containing the items in `q1` followed by the items in `q2`.

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

The Queue Interface and Its Use (3 of 5)

- Table 8.1 The Methods in the Queue Interface (continued)

Queue Method	What It Does
`q.__eq__(anyObject)`	Same as `q == anyObject`. Returns `True` if `q` equals `anyObject` or `False` otherwise. Two queues are equal if the items at corresponding positions are equal.
`q.clear()`	Makes `q` become empty.
`q.peek()`	Returns the item at the front of `q`. *Precondition*: `q` must not be empty; raises a `KeyError` if the queue is empty.
`q.add(item)`	Adds `item` to the rear of `q`.
`q.pop()`	Removes and returns the item at the front of `q`. *Precondition*: `q` must not be empty; raises a `KeyError` if the queue is empty.

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

The Queue Interface and Its Use (4 of 5)

- Table 8.2 The Effects of Queue Operations

Operation	State of the Queue after Operation	Value Returned	Comment
`q = <Queue Type>()`			Initially, the queue is empty.
`q.add(a)`	`a`		The queue contains the single item `a`.
`q.add(b)`	`a b`		`a` is at the front of the queue and `b` is at the rear.
`q.add(c)`	`a b c`		`c` is added at the rear.
`q.isEmpty()`	`a b c`	`False`	The queue is not empty.
`len(q)`	`a b c`	`3`	The queue contains three items.
`q.peek()`	`a b c`	`a`	Return the front item on the queue without removing it.

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

The Queue Interface and Its Use (5 of 5)

- Table 8.2 The Effects of Queue Operations (continued)

Operation	State of the Queue after Operation	Value Returned	Comment
`q.pop()`	`b c`	`a`	Remove the front item from the queue and return it. `b` is now the front item.
`q.pop()`	`c`	`b`	Remove and return `b`.
`q.pop()`		`c`	Remove and return `c`.
`q.isEmpty()`		`True`	The queue is empty.
`q.peek()`		`exception`	Peeking at an empty queue throws an exception.
`q.pop()`		`Exception`	Trying to pop an empty queue throws an exception.
`q.add()`	`d`		`d` is the front item.

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Two Applications of Queues

- This section looks briefly at two applications of queues:
  - One involving computer simulations
  - One involving round-robin CPU scheduling

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Simulations (1 of 2)

- Computer simulations are used to study behavior of real-word systems
  - Especially when it is impractical or dangerous to experiment directly
- Examples
  - Simulation that mimics traffic flow on a busy highway
  - Simulation that helps managers schedule employees and monitor technology
- Simulation programs use a simple technique to mimic variability:
  - Suppose new customers are expected on average once every 4 minutes
  - Then, during each minute of simulated time, a program can generate a random number between 0 and 1
  - If the number is less than $\frac{1}{4}$, the program adds a new customer to a checkout line; otherwise, it does not

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Simulations (2 of 2)

- Simulations operate by manipulating queues:
  - Example: associate each service provider with a queue of service customers
  - Adds varying numbers of consumers to the queues and gives consumers at the head of the queue another unit of service
  - Once a consumer receives service, it leaves the queue and next consumer steps forward
- Use object-oriented methods to implement simulation programs:
  - Generates new customer objects as appropriate
  - Assigns customers to cashiers
  - Tells each cashier to provide one unit of service to the customer at the head of the queue

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

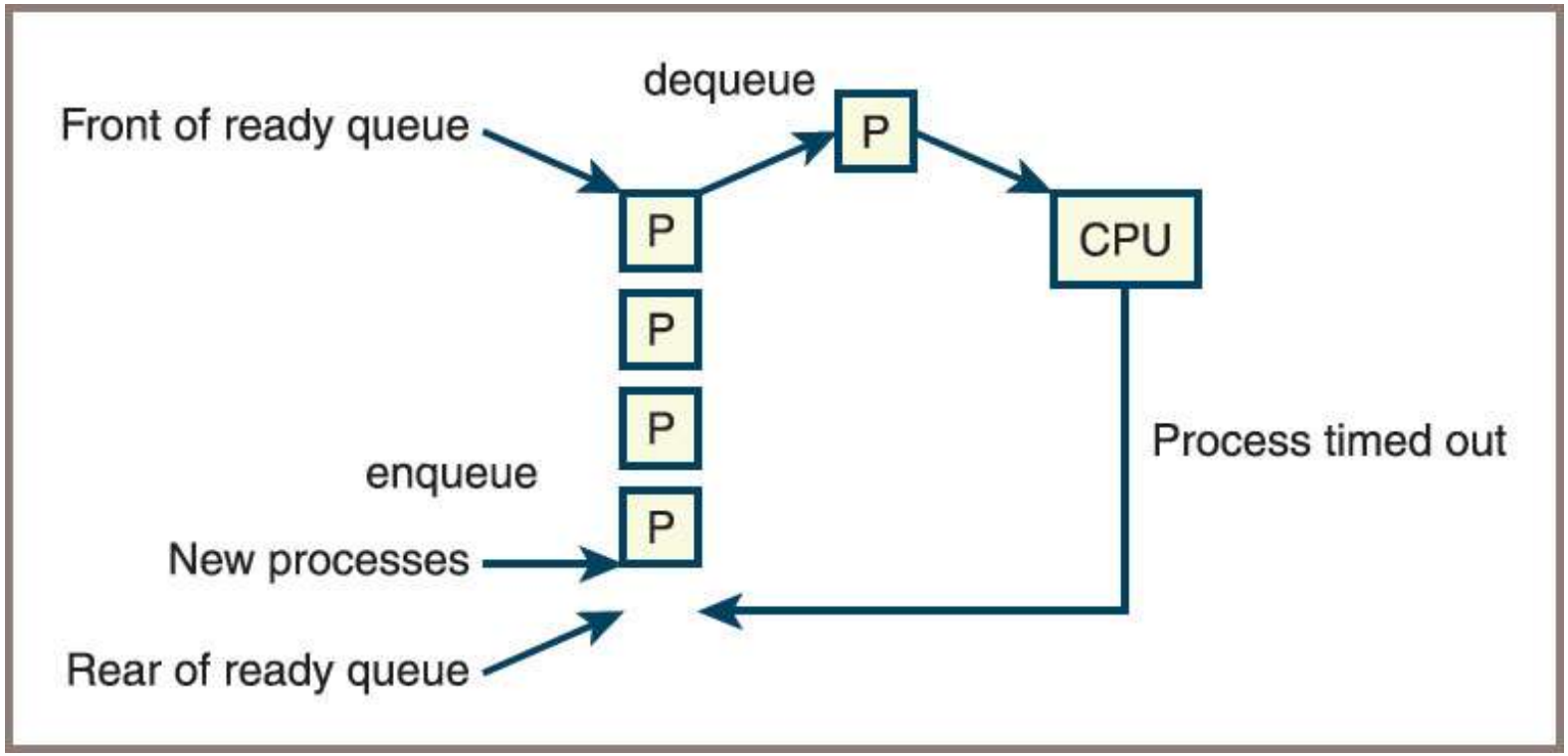
Round-Robin CPU Scheduling (1 of 2)

- Round-robin scheduling
  - Adds new processes to the end of a read*y queue*
    - Consists of processes waiting to use the CPU
  - Each process on the ready queue is popped in turn and given a slice of CPU time
  - When the time slice runs out, the process is returned to the rear of the queue

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Round-Robin CPU Scheduling (2 of 2)

Figure 8-2: Scheduling processes for a CPU



Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Implementations of Queues

- The structure of a queue lends itself to either an array implementation or a linked implementation:
  - Similar to approaches used for stacks in the previous chapter

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

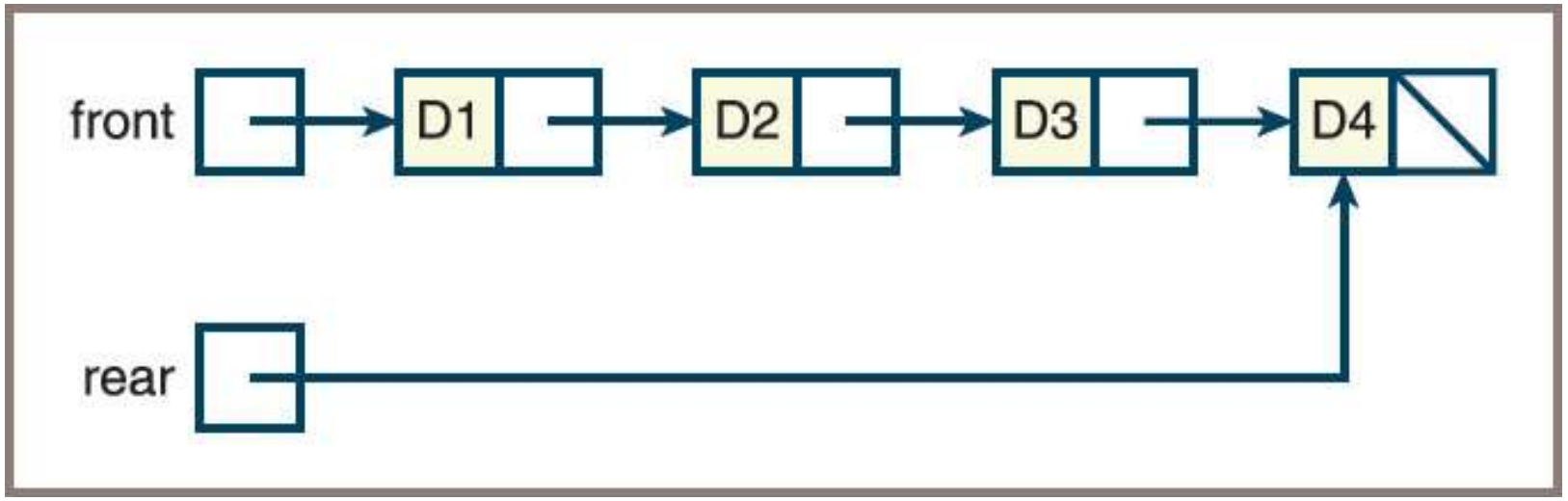
A Linked Implementation of Queues (1 of 6)

- Both **`LinkedStack`** and **`LinkedQueue`** classes
  - Use a singly linked **`Node`** class to implement nodes
  - The operation **`pop`** removes the first node in the sequence in both collections
- **`LinkedQueue.add`** and **`LinkedStack.push`** differ:
  - **`push`** adds a node at the head of the sequence
  - **`add`** adds a node at the tail
- To provide fast access to both ends of a queue's linked structure,
  - There are external pointers to both ends

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

A Linked Implementation of Queues (2 of 6)

Figure 8-3: A linked queue with four items



Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

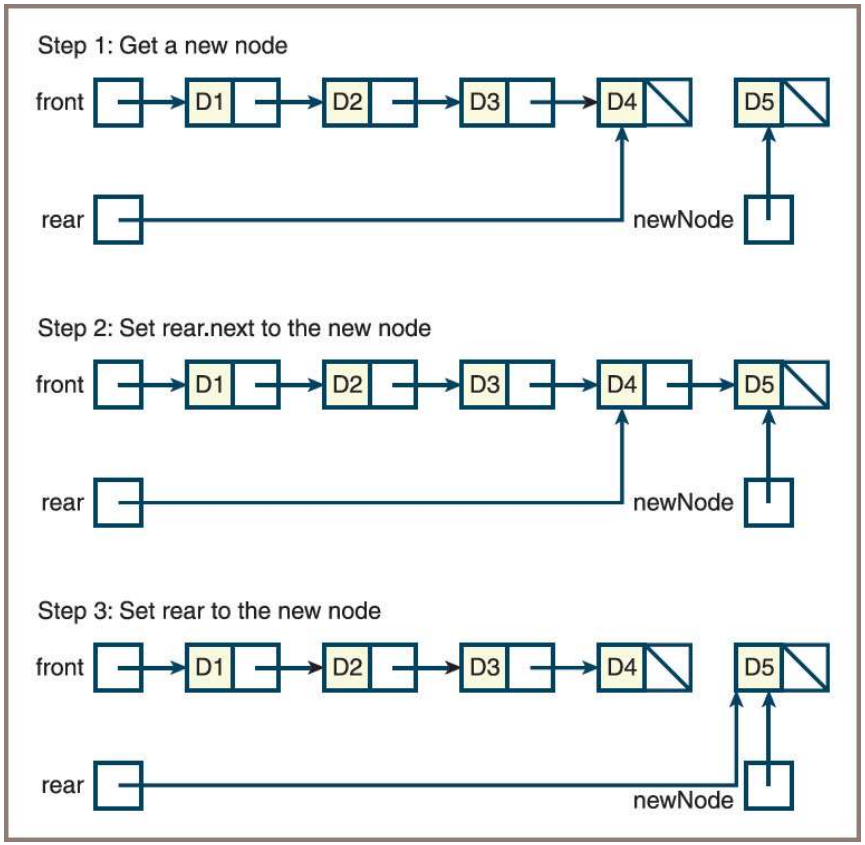
A Linked Implementation of Queues (3 of 6)

- The instance variables **`front`** and **`rear`** of the **`LinkedQueue`** class are given an initial value of **`None`**
- A variable named **`size`**, already defined in the collection framework, tracks the number of elements currently in the queue
- During an **`add`** operation,
  - Create a new node
  - Set the next pointer of the last node to the new node
  - Set the variable **`rear`** to the new node

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

A Linked Implementation of Queues (4 of 6)

Figure 8-4: Adding an item to the rear of a linked queue



Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

A Linked Implementation of Queues (5 of 6)

- Code for the add method:

```
def add(self, newItem):
    """Adds newItem to the rear of the queue."""
    newNode = Node(newItem, None)
    if self.isEmpty():
        self.front = newNode
    else:
        self.rear.next = newNode
    self.rear = newNode
     self.size += 1
```

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

A Linked Implementation of Queues (6 of 6)

- If the queue becomes empty after a **`pop`** operation, the **`front`** and **`rear`** pointers must both be set to **`None`**
- Here is the code:

```
def pop(self):
    """Removes and returns the item at front of the queue.
    Precondition: the queue is not empty."""
    # Check precondition here
    oldItem = self.front.data
    self.front = self.front.next
    if self.front is None:
        self.rear = None
    self.size -= 1
     return oldItem
```

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

An Array Implementation (1 of 6)

- The array implementation of a queue must access items at the logical beginning and the logical end
- Best to approach the problem in a sequence of three attempts
- First attempt
  - Fixes the front of the queue at index position 0 and maintains an index variable, called **`rear`**, that points to the last item at position $n - 1$
  - See Figure 8.5

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

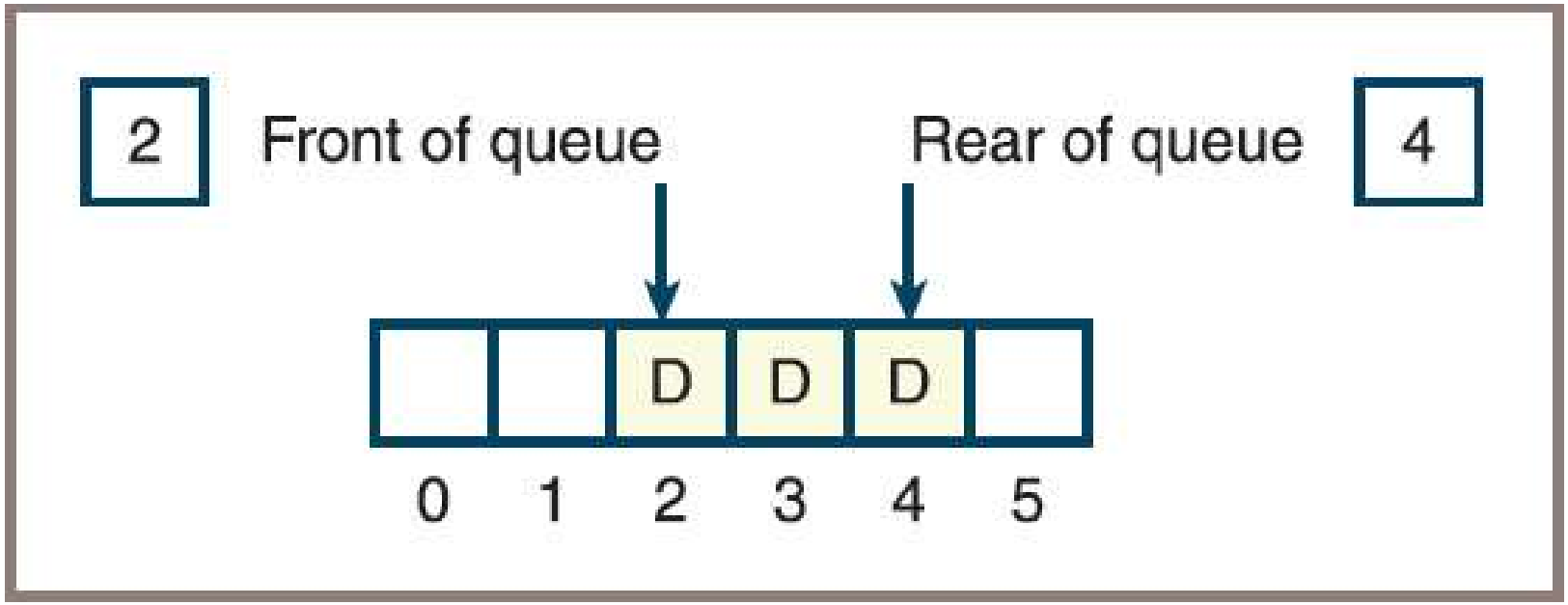
An Array Implementation (2 of 6)

- Second attempt
  - You can avoid `pop`'s linear behavior by not shifting items left each time the operation is applied
  - The modified implementation maintains a second index, called `front`, that points to the item at the front of the queue
  - The `front` pointer starts at 0 and advances through the array as items are popped
  - Figure 8.6 shows such a queue after five `add` and two `pop` operations

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

An Array Implementation (3 of 6)

Figure 8-6: An array implementation of a queue with a front pointer



Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

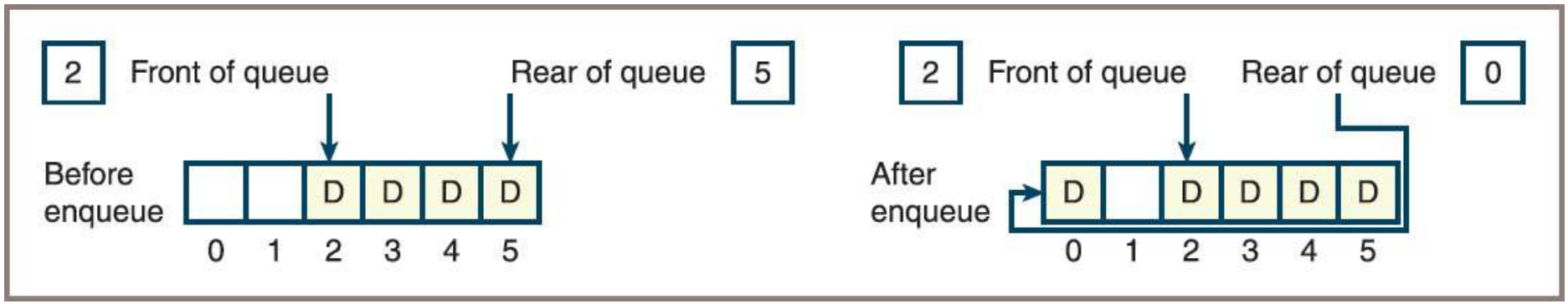
An Array Implementation (4 of 6)

- Third attempt
  - By using a *circular array implementation*, you can simultaneously achieve good running times for both **`add`** and **`pop`**
  - The implementation resembles the previous one in one respect: the **`front`** and **`rear`** pointers both start at the beginning of the array
  - See Figure 8.7

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

An Array Implementation (5 of 6)

Figure 8-7: Wrapping data around a circular array implementation of a queue



Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

An Array Implementation (6 of 6)

- After resizing, you would like the queue to occupy the initial segment of the new array with the **`front`** pointer set to 0
- Perform the following steps:
  - Create a new array that is twice the size of the current array
  - Iterate through the queue, using its **`for`** loop to copy items to the new array, starting at position 0 in that array
  - Reset the **`items`** variable to the new array
  - Set **`front`** to 0 and **`rear`** to the queue's length minus 1

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Time and Space Analysis for the Two Implementations

- The time and space analysis for the two queue classes parallels that for the corresponding stack classes
- Linked implementation of queues
  - The running time of the `__str__` , `__add__` , and `__eq__` methods is O(*n*)
  - The maximum running time of all the other methods is O(1)
- For the circular array implementation of queues,
  - If the array is static, the maximum running time of all methods other than `__str__` , `__add__` , and `__eq__` is O(1)

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Priority Queues (1 of 11)

- Priority queues
  - Items added to the queue are assigned an order of rank
  - Items of higher priority are removed before those of lower priority
  - Items of equal priority are removed in FIFO order
- Integers, strings, or any other objects that recognize comparison operators can be ordered in priority queues
- If an object does not recognize these operators,
  - It can be wrapped, or bundled, with a priority number in another object that does recognize these operators
  - The queue will then recognize this object as comparable with others of its type

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Priority Queues (2 of 11)

- Table 8.6 States in the Lifetime of a Priority Queue

Operation	State of Queue after Operation	Value Returned	Comment
`q = <Priority queue type>()`			Initially, the queue is empty
`q.add(3)`	3		The queue contains the single item 3
`q.add(1)`	1 3		1 is at the front of queue and 3 is at the rear of queue because 1 has higher priority
`q.add(2)`	1 2 3		2 is added but has a higher priority than 3, so 2 moves ahead of 3
`q.pop()`	2 3	1	Remove the front item from the queue and return it; 2 is now the front item
`q.add(3)`	2 3 3		The new 3 is inserted after the existing 3, in FIFO order
`q.add(5)`	2 3 3 5		5 has the lowest priority, so it goes to rear

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Priority Queues (3 of 11)

- You define a *wrapper class* used to build a comparable item from one that is not already comparable
- This new class is named **`Comparable`**:
  - It includes a constructor that expects an item and the item's priority as arguments
  - The priority must be an integer, a string, or another object that recognizes the comparison operators
- Code for the **`Comparable`** class:

```
class Comparable(object):
    """Wrapper class for items that are not comparable."""

    def __init__(self, data, priority=1):
        self.data = data
        self.priority = priority

    def __str__(self):
    """Returns the string rep of the contained datum."""
      return str(self.data)
```

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Priority Queues (4 of 11)

- Code for the **`Comparable`** class (continued):

```
def __eq__(self, other):
    """Returns True if the contained priorities are
equal
     or False otherwise."""
    if self is other: return True
    if type(self) != type(other): return False
    return self.priority == other.priority

 def __lt__(self, other):
    """Returns True if self's priority < other's
    priority,
    or False otherwise."""
  return self.priority < other.priority

def __le__(self, other):
    """Returns True if self's priority <= other's
    priority,
    or False otherwise."""
    return self.priority <= other.priority

def getData(self):
    """Returns the contained datum."""
  return self.data

def getPriority(self):
    """Returns the contained priority."""
   return self.priority
```

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Priority Queues (5 of 11)

- During insertions, a priority queue does not know whether it is comparing items in wrappers or just items
- When a wrapped item is accessed with the method `peek` or `pop` or in the context of a `for` loop,
  - It must be unwrapped with the method `getItem` before processing
- For example, assume that the items labeled `a`, `b`, and `c` are not comparable but should have the priorities 1, 2, and 3, respectively, in a queue:
  - Write code to add them to a priority queue named `queue` and retrieve them from it
  - See code on next slide

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Priority Queues (6 of 11)

- Code:

```
queue.add(Comparable(a, 1))
queue.add(Comparable(b, 2))
queue.add(Comparable(c, 3))
while not queue.isEmpty():
    item = queue.pop().getItem()
  <do something with item>
```

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Priority Queues (7 of 11)

- Sorted list
  - A list of comparable elements that are maintained in a natural order
- A priority queue's list should be arranged so that the minimum element is always accessed at or removed from just one end of the list:
  - The elements are inserted in their proper places in the ordering
- A singly linked structure represents this type of list well if the minimum element is always removed from the head of the structure
- If this structure is inherited from the singly linked structure used in the **`LinkedQueue`** class, you can continue to remove an element by running that class's **`pop`** method:
  - Only the **`add`** method needs to change; its definition is overridden in the new subclass, called **`LinkedPriorityQueue`**

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

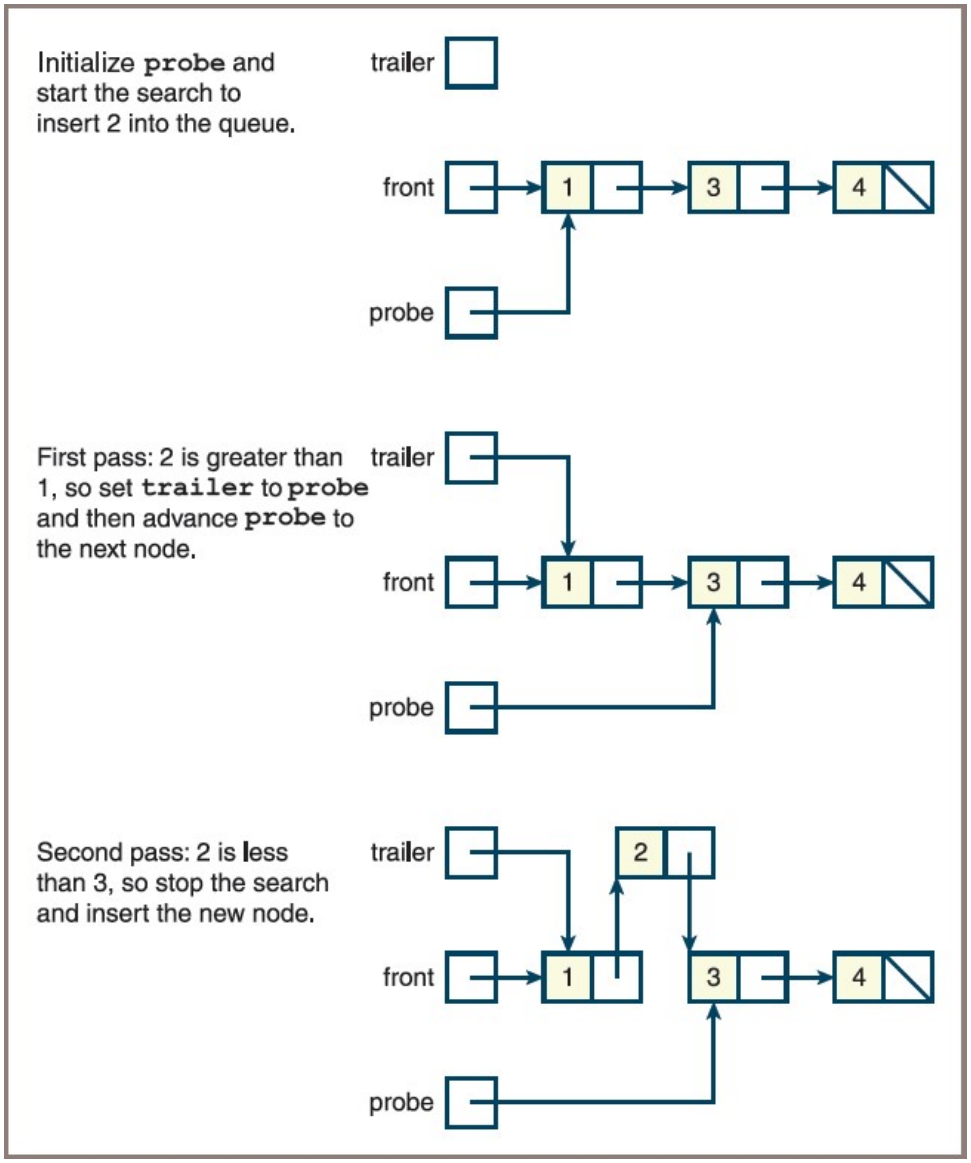
Priority Queues (8 of 11)

- The new implementation of **`add`** conducts a search for the new item's position in the list
- It considers the following cases:
  - If the queue is empty or the new item is greater than or equal to the item at the rear, add it as before (It will be placed at the rear)
  - Otherwise, begin at the head and move forward through the nodes until the new item is less than the item in the current node (At that point, a new node containing the item must be inserted between the current node and the previous node, if there is one)
- Figure 8.10 depicts the state of a priority queue containing the three integers 1, 3, and 4 during the **`add`** of the value 2

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Priority Queues (9 of 11)

Figure 8-10: Inserting an item into a priority queue



Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Priority Queues (10 of 11)

- Code for the class **LinkedPriorityQueue**:

```
"""
File: linkedpriorityqueue.py
"""

from node import Node
from linkedqueue import LinkedQueue

  class LinkedPriorityQueue(LinkedQueue):
   """A link-based priority queue implementation."""

  def __init__(self, sourceCollection = None):
      """Sets the initial state of self, which includes
   the
          contents of sourceCollection, if it's present."""
        LinkedQueue.__init__(self, sourceCollection)
```

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Priority Queues (11 of 11)

- Code for the class **`LinkedPriorityQueue`**: (continued)

```
def add(self, newItem):
    """Inserts newItem after items of greater or equal
    priority or ahead of items of lesser priority.
    A has greater priority than B if A < B."""
     if self.isEmpty() or newItem >= self.rear.data:
        # New item goes at rear
        LinkedQueue.add(self, newItem)
    else:
        # Search for a position where it's less
        probe = self.front
        while newItem >= probe.data:
            trailer = probe
            probe = probe.next
        newNode = Node(newItem, probe)
         if probe == self.front:
            # New item goes at front
            self.front = newNode
        else:
            # New item goes between two nodes
            trailer.next = newNode
      self.size += 1
```

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Chapter Summary (1 of 2)

- A queue is a linear collection that adds elements to one end, called the rear, and removes them from the other end, called the front
- Other operations on queues include peeking at the top element, determining the number of elements, determining whether the queue is empty, and returning a string representation
- Queues are used in applications that manage data items in a FIF O order
- Arrays and singly linked structures support simple implementations of queues

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.

Chapter Summary (2 of 2)

- Priority queues schedule their elements using a rating scheme as well as a FIFO order
- If two elements have equal priority, they are scheduled in FIF O order:
  - Otherwise, elements are ranked from smallest to largest, according to some attribute, such as a number or an alphabetical content

Fundamentals of Python: Data Structures, Second Edition. © 2019 Cengage. May not be copied, scanned, or duplicated, in whole or in part, except for use as permitted in a license distributed with a certain product or service or otherwise on a password-protected website for classroom use.